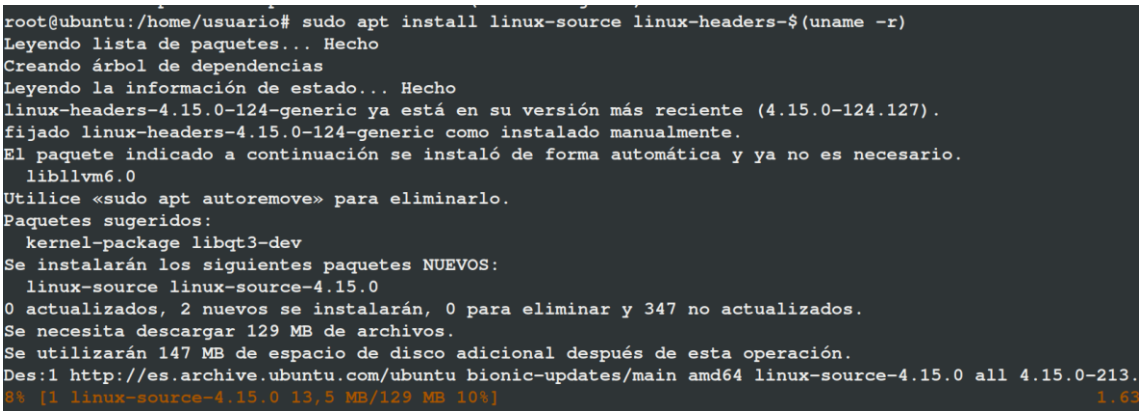
Solución practica 2.7

- sudo apt update && sudo apt upgrade -y
- Instala las herramientas necesarias:

```
sudo apt install build-essential bc bison flex libssl-dev
libncurses-dev libelf-dev dwarves \
     fakeroot ccache rsync git wget curl
```

- Instala los headers y fuentes del kernel:

```
sudo apt install linux-source linux-headers-$(uname -r)
```

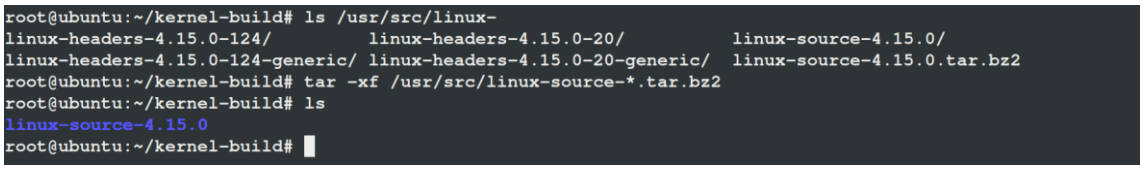


Nota: El código fuente se instala en /usr/src/linux-source-<versión>.tar.xz.

Crear directorio de trabajo y descomprimir fuentes

```
mkdir -p ~/kernel-build && cd ~/kernel-build
tar -xf /usr/src/linux-source-*.tar.xz
cd linux-source-*/
```

Nota: comprueba el nombre del archivo tar



Cargar configuración del kernel actual

- Copia la configuración del kernel que estás usando:

```
cp /boot/config-$(uname -r) .config
```

- Actualiza con las opciones nuevas:

```
yes "" | make oldconfig
(debes estar en el directorio  ~/linux-source-*/)
```

Contar componentes seleccionados

```
echo "Integrado (=y): $(grep -c '^[^#].*=y' .config)"
echo "Módulos (=m):   $(grep -c '^[^#].*=m' .config)"
echo "Total:          $(grep -c '^[^#].*=' .config)"
```

Ajustar configuración a los módulos cargados

```
make localmodconfig
```

Esto reduce la configuración a lo que realmente está cargado en tu sistema.

Volver a contar componentes

Repite el conteo:

echo "Integrado (=y): $(grep -c '^[^#].*=y' .config)"

echo "Módulos (=m): $(grep -c '^[^#].*=m' .config)"

echo "Total: $(grep -c '^[^#].*=' .config)"

Primera compilación y empaquetado

- Calcula núcleos de CPU:
- nproc
- Compila y genera paquetes .deb:
- make -j$(nproc) bindeb-pkg
- Los .deb se generan en el directorio superior (~/kernel-build).

Instalar el nuevo kernel

cd ..

sudo dpkg -i linux-image-*.deb linux-headers-*.deb

sudo update-grub

sudo reboot

En GRUB selecciona tu nuevo kernel y comprueba que arranca bien.

Reducir tamaño del kernel

- Copia la configuración anterior:

```
cp /boot/config-$(uname -r) .config
```

- Limpia archivos temporales:

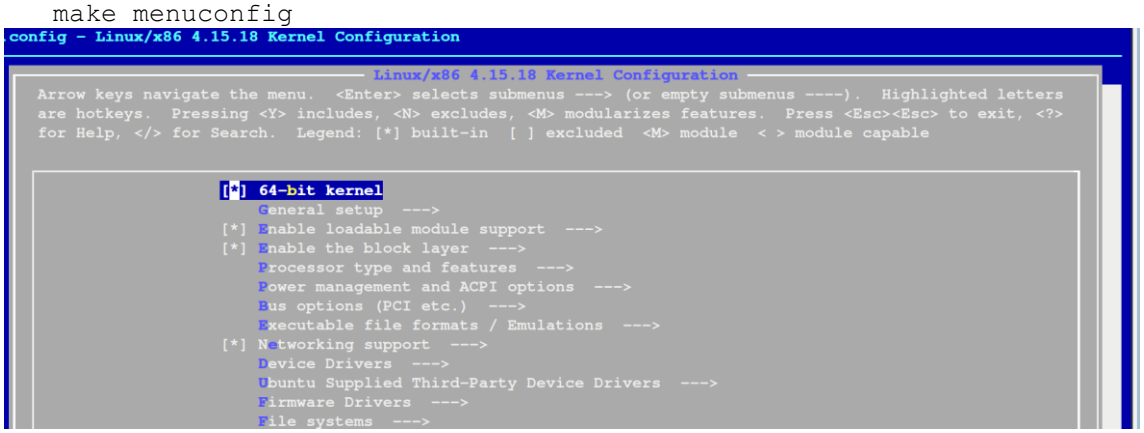
```
make clean
```

- Abre interfaz gráfica de configuración:

```
OPCION1: (interfaz qt)
sudo apt install qtbase5-dev qtchooser qt5-qmake qtbase5-dev-tools
make xconfig


OPCION2: (interfaz ncurses)

sudo apt install libncurses-dev
make menuconfig
```



Aquí desactiva lo que no uses: drivers de hardware que no tengas, sistemas de archivos que no uses, etc.

Volver a contar componentes

echo "Integrado (=y): $(grep -c '^[^#].*=y' .config)"

echo "Módulos (=m): $(grep -c '^[^#].*=m' .config)"

echo "Total: $(grep -c '^[^#].*=' .config)"

Compilar de nuevo

make -j$(nproc) bindeb-pkg

Instalar y probar

cd ..

sudo dpkg -i linux-image-*.deb linux-headers-*.deb

sudo update-grub

sudo reboot

Iterar hasta lograr un kernel mínimo

- Si falla el arranque (kernel panic, no encuentra root fs), vuelve a make xconfig y activa:
    - Controlador del disco raíz (SATA/AHCI/NVMe).

- Sistema de archivos raíz (ext4, XFS…).
- Consola básica (fbcon, VT).
- Controlador de red y teclado.

Repite el ciclo hasta que el kernel sea lo más pequeño posible pero funcional.

✅ **Consejos finales en Ubuntu**

- **Nunca desactives el driver del disco raíz ni el sistema de archivos del root.**
- **Usa initramfs**: Ubuntu lo genera automáticamente, así puedes dejar algunos drivers como módulos.
- **Compara tamaños**:
- ls -lh /boot/vmlinuz-*
- **Registra todo**:
- script -a ~/kernel_compile.log

(Esto guarda todos los comandos y salidas en un log).